

Miway CreditCore — Existing Code Assessment Report

Step 1 of the CreditCore rebuild: inspection of the codebase as it stands today, before further structural changes. This report describes the system as it currently is — a mechanical project rename (LoanSystem — MiwayCreditCore, module names/app atom/DB names only) has already been applied and verified (92 tests, 0 failures); no business logic, schema, or architecture described below has changed yet.

1. Versions

Component	Version
Elixir	1.17.3 (compiled with Erlang/OTP 26)
Erlang/OTP	26 (erts-14.2.5.3)
Phoenix	~> 1.7 (phoenix_html 3.3.4, phoenix_live_view 0.20.17 installed but unused — see §6)
Ecto / ecto_sql	~> 3.10 (3.11.3 installed)
PostgreSQL	16.14 (Ubuntu 16.14-0ubuntu0.24.04.1), local cluster on port 5433
postgrex	0.17.5

Notable dependencies: `bcrypt_elixir` (password hashing), `plug_attack` (rate limiting), `nimble_totp` + `eqrcode` (TOTP 2FA + QR code), `timex` (date math), `decimal` (money arithmetic).

2. Existing Contexts and Schemas

Context	Schemas	Responsibility
Accounts	<code>Accounts.User</code>	Authentication: email/password, <code>bcrypt</code> hash, role (admin/client), TOTP 2FA fields, lockout fields (failed_attempts, locked_until)
Clients	<code>Clients.Client</code>	Borrower records: name, phone, id_number, email, address, plus denormalized total_loans / current_balance recalculated after every loan mutation
Loans	<code>Loans.Loan</code> ,	The core lending domain —

Context	Schemas	Responsibility
	Loans.Payment, Loans.Collateral	see \$9/\$10, this is the part being restructured
Loans.InterestCalculator	(pure module, no schema)	Amortization math: monthly payment, total interest, compound lump sum, effective annual rate
FraudDetector	(pure module, no schema)	Scores a loan application 0+ across six weighted signals (new client, no repayment history, large first loan, amount vs. historical average, recent rejection, suspiciously round amount) → low/medium/high risk bucket
AuditLogs	AuditLogs.AuditLog	Generic actor/action/target/metadata/IP trail; log/2 never raises (failed write can't block the calling request)
ArrearsScheduler	(GenServer, no schema)	Runs every hour (10s after boot), flips overdue Payment rows from pending → overdue; disabled in :test env
Reports	(no schema, reads Loan/Payment)	Portfolio summary, overdue list, payments-due-soon, CSV export

3. Current Migrations and Database Tables

13 migrations, in order:

1. create_clients — clients table
2. create_loans — loans table
3. create_payments — payments table
4. create_users — users table
5. add_payments_and_update_loans
6. add_loan_and_payment_indices
7. fix_payment_date_type_and_indexes (2026-03-16)
8. create_audit_logs — audit_logs table
9. add_risk_level_to_loans

10. `add_2fa_and_lockout_to_users`
11. `create_collaterals` — `collaterals` table
12. `make_payment_date_nullable`
13. `add_microsecond_precision_to_audit_logs`

All primary keys are UUIDs (`binary_id`), consistently, across every table. Foreign keys are all `binary_id` typed to match. This convention should carry forward unchanged into any new tables.

Current table shapes (fields only, see the domain-restructure plan for full detail): - `clients`: `name`, `phone`, `id_number`, `email`, `address`, `active`, `total_loans`, `current_balance` - `loans`: `amount`, `interest_rate`, `term_months`, `status`, `approved_at`, `next_payment_date`, `remaining_balance`, `purpose`, `risk_level`, `client_id` - `payments`: `amount`, `payment_date`, `due_date`, `status`, `loan_id` - `collaterals`: `type`, `description`, `estimated_value`, `loan_id` - `users`: `email`, `password_hash`, `role`, `totp_secret`, `totp_enabled`, `failed_attempts`, `locked_until` - `audit_logs`: `event`, `actor_id`, `actor_email`, `target_type`, `target_id`, `metadata`, `ip_address`, `inserted_at` (microsecond precision)

4. Authentication System

Session-cookie based (not token-based), with a genuinely solid layered design for a project this size:

- **Password policy**: min 12 characters, requires upper+lower+digit, hashed with `bcrypt` (`Bcrypt.hash_pwd_salt/1`), constant-time verify, and `Bcrypt.no_user_verify()` called on unknown-email login attempts specifically to prevent timing-based user enumeration.
- **Account lockout**: 5 failed attempts locks the account for 15 minutes (`Accounts.authenticate_user/2`).
- **Rate limiting**: `PlugAttack` throttles `POST /login` to 10 attempts/minute/IP — a second, independent layer against credential stuffing across many accounts from one source, complementing the per-account lockout.
- **2FA**: TOTP via `nimble_totp`, QR provisioning via `eqrcode`, stored as a base64-encoded secret; login becomes a two-step flow (`pending_2fa_user_id` held in session) when `totp_enabled`.
- **CSRF**: `protect_from_forgery` in the browser pipeline.
- **CSP**: a real, restrictive content-security-policy header (`default-src 'self'`, no inline scripts, explicit allowances only for Google Fonts).
- **Session config**: cookie store, `same_site: "Lax"`, session renewed on login (`configure_session(conn, renew: true)` — correct session-fixation defense).
- **Production hardening**: `force_ssl`, `secret_key_base/DATABASE_URL/PHX_HOST` all required from environment variables at boot (`config/runtime.exs` raises loudly if missing — no silent insecure fallback), `Postgres` connection uses `ssl: true` in prod.

- **Authorization:** role gating via EnsureRolePlug (admin / client), checked per-route-scope in the router.

This is meaningfully more mature than a typical student project's auth layer — worth preserving as-is.

5. Existing Users, Customers, Loans, and Payments

- `priv/repo/seeds.exs` and `priv/repo/seeds/add_test_accounts.exs` create exactly two auth accounts: `admin@example.com` / `client@example.com`. No loans, clients, or payments.
- `priv/repo/test_seed.exs` is a separate, more elaborate **optional** script (not run automatically) that seeds four sample clients (Alice Banda, John Mwale, Grace Phiri, Peter Zulu) and five sample loans across pending/approved/completed/rejected states, plus two payments — clearly demonstration/QA data (Zambian names/phone formats, ZMW amounts), not real customer records.
- **Confirmed with the project owner:** nothing in the current database is real customer or transaction data. This is why the upcoming domain restructure is being done as a fresh-slate schema replacement rather than a data-preserving migration.

6. Router and LiveView Structure

Pure server-rendered controller + `.heex/.eex` templates — **no LiveView views actually exist**, despite `Phoenix.LiveView` being a dependency and the router importing `Phoenix.LiveView.Router` and mounting a `/live` socket in `endpoint.ex`. Zero live `"/... routes` are declared, and no module in the codebase uses `Phoenix.LiveView`. This is dead scaffolding today — either intentionally reserved for future interactive UI, or a leftover from `phx.new` defaults. Flagged under \$11 (Missing/Unused Architecture) rather than removed outright, since it's cheap to leave in place if LiveView is on the near-term roadmap.

Route map: `public (/ , /login, 2FA verify)`, rate-limited login POST, `/admin/*` (clients, loans, approve/reject, payments, collateral, reports, audit logs, 2FA settings — role: admin), `/client/*` (dashboard, read-only loans index/show — role: client).

7. Existing Tests

965 lines across 10 files, 92 tests + 1 doctest, **all passing** post-rename:

File	Lines	Covers
<code>loans_test.exs</code>	234	Create/approve/reject loan, fraud guards, payment creation + balance math, schedule generation, arrears
<code>accounts_test.exs</code>	170	Auth, logout, 2FA
<code>clients_test.exs</code>	136	Client CRUD + validations
<code>collateral_test.exs</code>	89	Collateral CRUD

File	Lines	Covers
fraud_detector_test.exs	80	All six fraud signals
arrears_test.exs	77	Overdue-flip logic
reports_test.exs	73	Portfolio summary, CSV export
audit_logs_test.exs	58	Audit log writing/reading
interest_calculator_test.exs	40	Amortization formula
miway_credit_core_test.exs	8	Placeholder app test

This is real, meaningful coverage for a project this size — a genuine asset going into the restructure, not something to discard. The upcoming domain split will require rewriting `loans_test.exs` proportionally (it's the one file that spans all five concepts being separated) while the other nine files are largely unaffected.

8. Security Weaknesses Found

Ranked by what actually matters, not by count:

1. **IDOR risk pattern present but currently mitigated at only one call site.**
`ClientLoanController.show/2` correctly checks `loan.client_id != client.id` before rendering — good. But this check is manual and per-controller, not structural (e.g., not enforced by the query itself or a shared plug). Any new client-facing read added later has to remember to add the same check by hand, and nothing would catch it if a developer forgot. Worth turning into a query-level guard (`get_loans_for_client/1`-style scoping) rather than a fetch-then-compare-then-maybe-403 pattern, as new client-facing routes are added.
2. **User→Client linkage is by email string match, not a foreign key.** `client_for/1` in both client controllers calls `Clients.get_client_by_email(user.email)` and **raises a bare `RuntimeError`** (`raise "No client record found for user ..."`) if no match exists — this is an unhandled-exception 500, not a graceful error page, for any client-role user whose email doesn't exactly match a `Client` row (e.g., created via `Accounts.create_user/1` without a corresponding `Client`, or an email typo in either table). This is both a correctness gap and a minor DoS/robustness concern. A real FK (`user_id` on `Client`, or vice versa) closes this properly.
3. **No visible user-registration or admin-user-creation route.**
`Accounts.create_user_with_role/1` exists but nothing in the router/controllers calls it — the only way to create a `User` today is `seeds.exs` or a direct `iex/Mix` task invocation. Not a vulnerability per se, but a real gap if admins are expected to onboard new staff/client logins through the app itself rather than ops tooling.

4. **Session cookie `signing_salt/encryption_salt` are hardcoded literals** (`"loan_sys_sign" / "loan_sys_enc"` in `endpoint.ex`) rather than derived from `secret_key_base`-adjacent config. This is **not actually a weakness** in Phoenix's design — these salts are combined with `secret_key_base` via HKDF and aren't meant to be secret on their own — but the literal strings still read as a legacy artifact of the old app name and are worth renaming for consistency now that the project is Miway CreditCore, even though there's no security fix needed here.
5. **No brute-force protection on the 2FA code entry itself** (`TwoFactorController.confirm/2`) — `PlugAttack` only throttles `POST /login`, not `POST /login/verify`. A 6-digit TOTP code has a 30-second validity window and ~1,000,000 possibilities; without rate limiting, an attacker who has already obtained a valid password (e.g., via a leak) could attempt to brute-force the current TOTP window. Low likelihood given the 30s window, but cheap to close with the same `PlugAttack` pattern already used for login.
6. **No rate limiting on 2FA setup/enable** (`POST /admin/settings/2fa/enable`) — same class of gap as above, lower severity since it's admin-only and requires an authenticated session already.

No SQL injection surface found (no raw `Ecto.Adapters.SQL.query` or unguarded `fragment/1` usage anywhere in `lib/`). No mass-assignment issues found — every `cast/3` call has an explicit, deliberately scoped field allowlist; role assignment is only castable via the separate `admin_changeset/2`, never the general-purpose `changeset/2`, so a client can't self-promote to admin through a form.

9. Code That Can Remain As-Is

- **Accounts / Accounts.User** — the entire auth system (§4). Solid, no changes planned or needed as part of the domain restructure.
- **AuditLogs** — generic, already decoupled from the specific shape of loans/payments; just gets called with new `target_type` values (`"loan_application"`, `"loan_account"`, `"payment_transaction"`) going forward.
- **Clients / Clients.Client** — the client/borrower record itself is fine; only its two *denormalized* fields (`total_loans`, `current_balance`) need their recalculation logic repointed at the new tables (see the restructure plan, §“Client denormalized stats”).
- **Loans.InterestCalculator** — the amortization math is correct and self-contained; only its call sites change (which struct's fields get passed in), not the formula.
- **ArrearsScheduler** — the `GenServer` scheduling mechanism is fine; only the context function it calls gets renamed.
- **Router pipelines, plugs, CSP, rate limiting** — all of §4 stays exactly as it is.
- **The test suite's structure and conventions** (`DataCase`, fixtures pattern) — being extended, not replaced.

10. Code Requiring Correction

This is the reason the restructure was requested — see the accompanying domain-restructure plan for full detail. In short:

- **Loans . Loan** conflates the loan *application* (requested amount, purpose, pending/rejected decision, risk scoring) with the loan *account* (approved_at, remaining_balance, next_payment_date, completed status) in one table/schema.
- **Loans . Payment** conflates the *repayment schedule* (bulk-inserted planning rows, due_date set, flipped to overdue by the scheduler) with *actual payment transactions* (independently inserted when money is recorded as received, payment_date set) — **with no foreign key linking a transaction to the installment it satisfies.**
- **remaining_balance** is a mutable field, decremented imperatively inside create_payment/1's transaction, with **no ledger** — no way to reconstruct how the balance arrived at its current value, no audit trail of the account's financial history beyond the generic AuditLogs action metadata.
- **Collateral timing:** today Collateral attaches to a Loan regardless of status, meaning collateral can be pledged before a loan decision is made — confirmed with the project owner that this should change to post-approval-only attachment once LoanAccount exists.
- The IDOR-pattern and User→Client-linkage issues from §8 (items 1–2) are worth fixing in the same pass as the controller rewiring the restructure already requires, since those controllers are being touched anyway.

11. Missing Architecture

- **No accounting/ledger layer** — the single biggest structural gap, and the primary driver of the restructure (see §10, and the separate five-entity restructure plan for the full design: LoanApplication, LoanAccount, RepaymentScheduleInstallment, PaymentTransaction + PaymentAllocation, AccountingEntry).
 - **No payment-to-installment allocation** — a transaction and the schedule row it pays down are currently unrelated records.
 - **No transaction voiding/reversal path** — a mis-recorded payment has no defined correction path today beyond direct DB editing; the restructure plan adds void_payment/2 with a compensating ledger entry.
 - **Unused LiveView scaffolding** (§6) — either commit to it for a future interactive feature (live payment recording, live dashboard updates) or strip the unused dependency/socket/import to reduce surface area. Not urgent either way.
 - **No user-registration/admin-user-creation UI** (§8, item 3) — every account today is created outside the running application.
 - **No rate limiting on the 2FA verification/enable endpoints** (§8, items 5–6).
-

Prepared as Step 1 of the Miway CreditCore rebuild. Step 2 (the five-entity domain restructure — LoanApplication / LoanAccount / RepaymentScheduleInstallment / PaymentTransaction /

AccountingEntry) is planned separately and is already underway following the project rename covered in \$1's version-and-rename note.